

Proof of Permission

A Compliance-Native Architecture for AI-Driven Financial Systems

Deterministic authorization, machine-readable legal capability, bounded execution authority, canonical ledger truth and reconstructable evidence for financial systems in which AI can reason and propose without becoming the legal or monetary principal.

THE CONTROL DOCTRINE

AI may discover and propose. Policy determines permission. Risk determines exposure. Controlled financial infrastructure executes. The ledger establishes truth. The Evidence Vault proves what happened.

Technical / Regulatory Architecture White Paper

September 2026 | SunRey

Conceptual architecture. This paper describes a technical control model; it does not itself establish legal permission, licensing, regulatory approval, or production authorization in any jurisdiction.

Executive Abstract

Financial AI creates a control problem before it creates a model problem. A system may be capable of discovering a beneficial transfer, investment, payment route, data use, or digital-asset action and still lack the legal, customer, risk, jurisdictional, operational, or evidentiary authority to execute it. SunRey therefore treats intelligence and authority as separate architectural domains.

Proof of Permission is the control spine for that separation. Every consequential human- or AI-initiated proposal becomes an ActionIntent. The Compliance Kernel evaluates a bounded set of proofs and versioned policies. Only a signed, short-lived Execution Authority may authorize ledger-impacting execution. The resulting financial event is then recorded in the canonical ledger and sealed with the decision context needed to reconstruct why it was allowed, conditioned, escalated, deferred, or blocked.

SFF 2.0 extends that control spine with a Regulatory Digital Twin: a machine-readable representation of what the platform is legally and operationally capable of doing for a particular product, customer, jurisdiction, legal entity, partner, license state, control requirement, policy version and counsel review state. The objective is not to automate legal judgment away. It is to make approved legal and control judgments enforceable, testable and auditable in software.

CENTRAL CLAIM

The differentiator is not that AI can participate in finance. It is that AI-generated economic action can be translated into a deterministic authorization and evidence chain before execution.

White Paper Structure

This paper develops Proof of Permission as a compliance-native systems architecture rather than as a checklist layered onto product logic after the fact. The sections move from the control problem, through the six foundational proofs, into execution authority, accounting truth, evidence, jurisdictional capability and institutional operation.

Section	Theme	Purpose
01-04	Thesis & Problem	Why intelligence, recommendation and execution authority must be separated.
05-08	Control Spine	ActionIntents, Compliance Kernel and the Proof of Permission model.
09-14	Six Proofs	Identity, Authority, Jurisdiction, Compliance, Risk and Purpose.
15-18	Execution	Extended control dimensions, deterministic outcomes and signed Execution Authority.
19-22	Truth & Evidence	Financial fabrics, ledger invariants, proof-carrying events and the Evidence Vault.
23-27	Regulatory Runtime	Regulatory Digital Twin, capability state, Jurisdiction Packs and legal-entity resolution.
28-31	Bounded AI & Examples	Sovereign Cells, Agent Capability Tokens and reference transaction journeys.
32-36	Assurance & Operation	Testing, simulation, failure modes, audit reconstruction and institutional adoption.
37-38	Doctrine & References	Design principles, conclusion, terminology and source basis.

PUBLICATION NOTE

Earlier source documents use the working names Solstice and Pyramid. This paper uses the current SunRey brand while preserving the underlying control architecture and terminology where it remains technically relevant.

Why Compliance After Intelligence Is Too Late

Conventional software often treats compliance as a set of checks attached to an application workflow: call identity, sanctions, fraud or suitability services, then allow the product service to continue. That pattern becomes fragile when many AI agents, providers, markets, legal entities and jurisdictions can initiate or shape economically consequential decisions.

In an agentic system, the control boundary must be clearer. An AI can be excellent at discovering options while remaining an inappropriate holder of legal, monetary or customer authority. A provider can return a successful API response without having the right to determine whether the transaction should exist. A user can express a goal without necessarily authorizing every possible implementation of that goal.

The architectural response is to make permission a first-class object. The system asks not only whether an action is technically possible, but who is involved, who authorized it, what jurisdiction and legal actor govern it, whether the activity is permitted now, whether exposure is within deterministic limits, and whether any sensitive information is being used for an authorized purpose.

- Do not infer authorization from a model recommendation.
- Do not infer legal capability from software availability.
- Do not infer customer consent from possession of data.
- Do not infer risk permission from commercial desirability.
- Do not infer monetary authority from provider success.

DESIGN SHIFT

Compliance is not a downstream veto. It is part of the transaction state required before execution authority can exist.

Authority Separation as a System Property

Proof of Permission is built around a strict separation of roles. Intelligence proposes. Policy and control determine whether the proposal is admissible. Risk constrains exposure. A designated execution layer translates approved authority into action. Accounting systems establish financial truth. Evidence systems preserve why the action occurred.

This separation matters because financial systems combine several forms of authority that should not collapse into one another: user authority, legal-entity authority, regulatory capability, risk authority, model approval, operational authority, payment or custody provider authority, and ledger authority. The architecture makes those boundaries explicit rather than allowing a single application service to implicitly assume them.

The result is a control structure in which a model may become more capable without automatically becoming more powerful. Better reasoning can improve the quality of ActionIntents while the maximum authority of the model remains unchanged.

Proof of Permission - Authorization Spine

Intelligence may propose. Only explicit authority may execute.

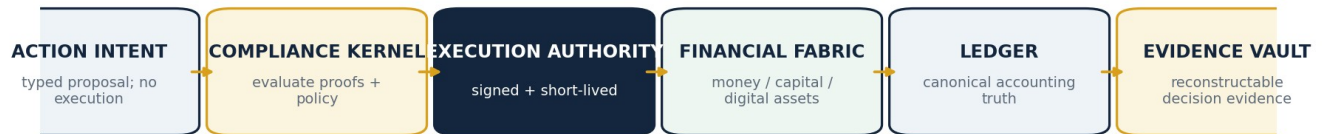


Figure 1. The Proof of Permission authorization spine separates proposal, permission, execution, accounting truth and evidence.

CONTROL PRINCIPLE

Capability can expand without authority expanding. This is the core safety property for AI-driven finance.

The End-to-End Authorization Chain

The canonical sequence is: ActionIntent -> Compliance Kernel -> Execution Authority -> Financial Fabric -> Ledger -> Evidence Vault. Each stage owns a different question and each transition is explicit.

An ActionIntent describes the requested economic action and its context. The Compliance Kernel resolves proof dimensions and policy. Execution Authority is a signed, short-lived artifact expressing the exact bounded action that may proceed. A domain-specific financial fabric performs the permitted operation. The canonical ledger records the resulting financial state. The Evidence Vault preserves the decision context and outcome.

The sequence is deliberately asymmetric. Intelligence can submit a proposal into the chain, but it cannot skip a stage. A successful compliance decision does not itself create a balance. A provider confirmation does not itself rewrite accounting truth. A ledger entry does not retroactively manufacture missing customer or legal authority.

Layer	Question owned	Authority boundary
ActionIntent	What is being proposed?	No execution authority.
Compliance Kernel	Is the action permitted under current proofs and policy?	May allow, condition, escalate, defer or block.
Execution Authority	What exact bounded action may execute?	Signed, scoped and short-lived.
Financial Fabric	How is the approved action operationally performed?	Domain execution only.
Ledger	What financial state actually exists?	Canonical accounting authority.
Evidence Vault	Why did the system act this way?	Decision reconstruction and audit evidence.

ActionIntent: The Boundary Object

An ActionIntent is the typed proposal that crosses from reasoning into control. It prevents recommendations, UI gestures, agent outputs and provider callbacks from being mistaken for executable financial instructions. Every consequential action should be normalized into an intent before authorization begins.

A robust intent carries enough information for the control plane to evaluate identity, authority, jurisdiction, compliance, risk and purpose without relying on hidden model state or unstructured conversation history. It should also be idempotent and uniquely identifiable so retries cannot duplicate economic effects.

ActionIntent is therefore not merely a message format. It is the architectural boundary between economic intelligence and economic authority.

Intent field family	Illustrative purpose
Actor / subject	Identify the person, organization, agent or service involved.
Action type + parameters	Describe the proposed transfer, trade, data use, approval or ledger-affecting operation.
Economic amount / scope	Define value, quantity, asset class, frequency, duration and account boundaries.
Source context / provenance	Identify the facts, recommendations, quotes or provider observations supporting the proposal.
Risk context	Carry applicable limits, mandate, exposure class and requested exceptions.
Approval requirements	State whether user approval, step-up authentication or manual review is required.
Idempotency / expiry	Prevent duplicate execution and stale authority.

BOUNDARY RULE

An ActionIntent is a proposal. It is not a transaction, an order, a transfer, an authorization or a ledger posting.

Proof of Permission: Six Independent Questions

The source architecture defines six foundational proofs. They are independent in the sense that success in one dimension does not imply success in another. A verified customer may still lack authority for a specific action. An authorized action may still be prohibited in the relevant jurisdiction. A legally available product may still exceed deterministic risk limits. Data may be available while its proposed use remains outside consented purpose.

The Kernel resolves these dimensions together and produces one deterministic outcome. Escalation is monotonic: posture may tighten during the request, but it should never silently relax because a downstream service is eager to proceed.

This makes permission compositional. The system can add new products, agents, providers and jurisdictions while reusing the same proof grammar.

The Six Foundational Proofs

Every consequential ActionIntent must resolve all six dimensions before execution authority can exist.

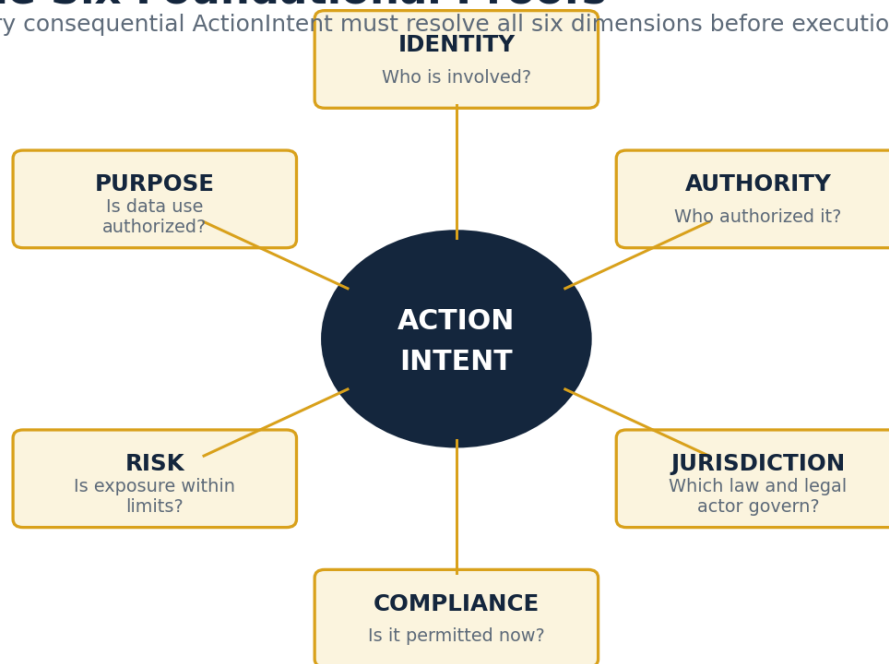


Figure 2. The six foundational proofs: Identity, Authority, Jurisdiction, Compliance, Risk and Purpose.

SIX-PROOF RULE

Every consequential action must have a resolved answer to all six questions before a signed Execution Authority can be issued.

Identity: Who Is Involved?

Identity proof asks who is involved in the action and whether the relevant identity state is current and verified. It is broader than a one-time KYC result. Financial authorization can depend on the current user, beneficial owner, device, account holder, organization, counterparty, agent identity and verification freshness.

The purpose of the proof is not to turn identity providers into execution authorities. Provider attestations are inputs. The Compliance Kernel decides how those attestations satisfy the current product and jurisdictional requirement.

Identity state should be versioned and evidentiary. If the system later needs to explain why an action was allowed, it must be possible to identify which identity state, verification level and relevant device or organizational attributes were relied upon at the moment of decision.

- Current identity state, not merely historical onboarding completion.
- Relevant actor and counterparty resolution.
- Step-up authentication where the action requires stronger assurance.
- Separation between identity evidence and financial execution authority.
- Evidence of the identity state used in the decision.

QUESTION

Who is involved, and is the identity state current and verified?

Authority: Who Authorized This Exact Action?

Authority proof asks who authorized the specific action and within what limits. This is where user mandates, organizational roles, account permissions, agent capability boundaries and manual approvals become operational rather than aspirational.

A broad goal such as "grow my money" or "send funds internationally" does not automatically authorize every trade, venue, amount, recipient or timing choice. Authority should be scoped to the exact category of action, the applicable limits, the validity window and any required approval threshold.

For AI systems, the rule is especially strict: a model or agent may have proposal authority without having customer, legal, governance or wallet authority. The system should never infer an agent's execution rights from its ability to generate a high-confidence recommendation.

Authority source	What it may establish	What it must not imply
Customer mandate	Permitted categories, limits, goals, time horizon and approval preferences.	Unlimited discretion outside mandate.
Role / organization	Operational or administrative rights for a named role.	Ownership of customer funds.
Agent capability token	What an AI service may propose and what it is forbidden to request.	Execution credentials.
Manual approval	Human approval of a bounded exceptional action.	Permanent policy override.
Execution Authority	The exact action allowed to execute now.	General future authority.

QUESTION

Who authorized this specific action, and within what limits?

Jurisdiction: Which Law, Legal Actor and Regional Cell Govern?

Jurisdiction proof resolves the legal and operational context in which the action exists. The source architecture treats this as more than a country code. The relevant state may include source and destination jurisdictions, product classification, named legal entity or regulated partner, regional deployment cell, customer location, asset or account location and corridor-specific requirements.

This prevents the platform from treating a technically available feature as globally enabled. A product path exists only where the applicable jurisdictional capability has been positively established and mapped to a responsible legal actor.

The architecture also supports regional Sovereign Cells so that data residency, encryption keys, local ledgers or partitions, policy packs, approved models and recovery controls can be enforced structurally.

- Resolve the governing legal entity or regulated partner.
- Resolve source and destination jurisdiction for cross-border activity.
- Bind the action to the correct Jurisdiction Pack and capability state.
- Respect cell-local data, key and model restrictions.
- Default unresolved or unconfirmed capability to disabled.

QUESTION

Which law, regulator, legal entity and regional operating boundary govern the action?

Compliance: Is the Product or Activity Permitted Now?

Compliance proof asks whether the product or activity is institutionally and legally permitted at the moment of execution, given the current control state. In the source architecture, this can incorporate KYC, AML, sanctions, product rules, transaction monitoring, disclosures, source-of-funds evidence, market rules and other domain-specific requirements.

The important architectural point is that compliance providers and product services do not own the final permission decision. They produce facts, screenings, risk indicators or required artifacts that the Kernel evaluates against versioned policy.

Because rules change, compliance state must be time-aware and versioned. A transaction should be reconstructable against the rules that were authoritative when it was processed rather than against whatever rules happen to be current during a later audit.

- Provider success is evidence, not permission.
- Policy versions must be captured with the decision.
- Missing required evidence can trigger a condition, refresh, manual review, defer or block.
- Capability remains disabled when legal or control state is unresolved.
- A later policy change must not rewrite the historical decision record.

QUESTION

Is this action legally and institutionally permitted right now?

Risk: Is Exposure Within Deterministic Limits?

Risk proof separates economic attractiveness from permission to assume exposure. A proposed trade, transfer, exchange action or credit-like commitment can appear beneficial while violating liquidity, leverage, concentration, drawdown, fraud, operational or product-specific limits.

The SunRey architecture therefore places deterministic risk controls above AI allocation or recommendation logic. Within the Agentic Capital Mesh, for example, specialized agents may search for opportunity and propose exposures, but the Risk Engine remains the final authority on leverage, concentration, liquidity, drawdown and kill-switch behavior.

Risk limits should be explicit, versioned and enforceable outside model prompts. If a model proposes an action beyond the allowed envelope, the correct response is not to persuade the model to behave. The control plane rejects or conditions the proposal.

Risk dimension	Control question
Exposure	Does the action fit amount, leverage and notional limits?
Liquidity	Can obligations and required reserves still be met?
Concentration	Does the action violate asset, issuer, venue, corridor or counterparty concentration?
Drawdown / loss	Would the action breach bounded loss or drawdown constraints?
Operational risk	Are provider, system and settlement states healthy enough to proceed?
Kill switches	Has a domain, model, account, provider or jurisdiction been suspended?

QUESTION

Is the financial exposure within permitted deterministic limits?

Purpose: Is Information Used Only for an Authorized Purpose?

Purpose proof ensures that possession of information never becomes permission to use it. The source architecture includes a Purpose Firewall and granular consent model so that a customer fact may be relevant to one decision yet prohibited for another.

This is especially important when AI agents operate over broad personal context. A system may know income, health, location, goals, subscriptions, employment or other sensitive attributes. The fact that such information exists in a protected graph or vault does not imply that every agent, model or product may use it.

Purpose should therefore be bound to the ActionIntent and evaluated before the system permits information-dependent economic action. The evidence record should preserve the purpose basis and the consent or policy object that supported it where required.

- Data relevance is not data permission.
- Purpose is explicit and action-specific.
- Permissions may be granular, time-limited and revocable.
- Sensitive information can remain isolated while proofs or permitted outputs cross the control boundary.
- Purpose and consent state belong in reconstructable evidence.

QUESTION

Is any personal or sensitive information being used only for an explicitly authorized purpose?

Model State and Consent as Explicit Control Dimensions

The expanded SFF 2.0 architecture retains the six foundational proofs but adds Model State and Consent as explicit subproofs or control dimensions for AI- and data-intensive actions. This prevents two common assumptions: that an available model is automatically approved for economically consequential use, and that application-level access to data implies a valid consent basis.

Model State asks whether the economically consequential model is approved, monitored and permitted for the specific product and jurisdiction. Consent asks whether a current, granular consent object covers the requested data use where required.

Treating these as explicit dimensions creates a more durable control model. An AI system can be upgraded, rolled back or disabled without rewriting product logic; consent can expire or be revoked without requiring each application service to independently remember the change.

Control dimension	Question	Effect
Model State	Is the model approved, monitored and allowed here?	Gates model eligibility by product, jurisdiction and lifecycle state.
Consent	Does a current granular consent object cover this use?	Gates data use and can expire or revoke independently.
Foundational six proofs	Are identity, authority, jurisdiction, compliance, risk and purpose resolved?	Remain the mandatory base authorization grammar.

ARCHITECTURAL POSITION

Model State and Consent extend the control envelope without weakening the six-proof foundation.

Deterministic Outcomes Instead of Binary Compliance

The Compliance Kernel need not reduce every control evaluation to a simplistic yes or no. The source architecture defines a bounded set of deterministic outcomes that allow the system to request stronger evidence, disclose conditions, route to human review or defer execution without losing the integrity of the control chain.

This is critical for operational finance because many transactions are resolvable rather than inherently prohibited. An expired KYC state may require refresh. A high-impact action may require step-up authentication. A corridor may require a disclosure. A suspicious condition may require source-of-funds evidence or manual review.

Every outcome, including a block, should be evidentiary. The platform can therefore answer not only "what executed?" but also "what was proposed and why was it stopped?"

Outcome	Operational meaning
ALLOW	Proceed to the next controlled stage.
ALLOW WITH CONDITIONS	Proceed only with explicit additional constraints.
STEP-UP AUTHENTICATION	Require stronger user authentication.
REQUIRE DISCLOSURE	Present required information before continuation.
REQUIRE KYC REFRESH	Update identity / eligibility evidence.
REQUIRE SOURCE-OF-FUNDS EVIDENCE	Collect additional transaction evidence.
REQUIRE MANUAL REVIEW	Route to an authorized human workflow.
DEFER	Do not execute now; await a resolvable condition.
BLOCK	Do not permit the action.

MONOTONIC ESCALATION

During a request, control posture may tighten. It must not silently relax because a downstream service is available or a model is confident.

Signed Execution Authority: The Only Executable Permission Artifact

Execution Authority is the bridge from permission to action. It is signed, short-lived and scoped to the exact approved transaction or operation. In the source architecture, the ledger will not accept customer-impacting financial movement without this artifact.

This design prevents authorization from being implicit in process memory, model output, user interface state or provider response. The execution layer receives a verifiable object defining what may happen now, under which parameters and within which constraints.

Execution Authority should expire quickly, bind to the ActionIntent and decision context, and resist replay. If transaction parameters change materially after authorization, the system should produce a new ActionIntent and re-evaluate permission rather than stretching the previous approval.

Property	Required behavior
Signed	Execution layer can verify provenance and integrity.
Short-lived	Authority expires rather than becoming an open-ended credential.
Intent-bound	The authorization references the exact ActionIntent and approved parameters.
Scope-limited	Asset, amount, account, recipient, product and/or action bounds are explicit.
Replay-resistant	Idempotency and unique identifiers prevent duplicate economic effects.
Revocable / condition-aware	Emergency controls or invalidated preconditions can stop execution.

AUTHORITY RULE

A policy decision is not a balance change. Only an authorized execution path may create the financial event that the ledger records.

Financial Fabrics Preserve Domain Boundaries

After authorization, execution passes into a domain-specific financial fabric rather than a generic "money mover." The source architecture distinguishes Money Fabric, Capital Fabric and digital-asset / data-related fabrics so that deposits, payments, investments and native assets do not collapse into one undifferentiated balance system.

This segregation supports different legal entities, accounting treatments, risk models, provider connections and settlement states while retaining one authorization spine. It also prevents the Exchange, an AI agent or a product service from creating a second balance authority outside the canonical ledger or chain owner.

Crossing account or asset classes should therefore be explicit. A movement from deposit cash into an investment position, for example, is a named and disclosed bridge rather than a silent transformation inside a consumer interface.

Fabric	Typical scope	Control emphasis
Money Fabric	Accounts, payments, cards, FX, cash management.	Deposit/account segregation, payment legality, reconciliation.
Capital Fabric	Investments, portfolios, orders, execution.	Suitability/mandate, market risk, model lifecycle, venue/execution policy.
Digital Asset / Exchange Fabric	Wallets, custody, exchange and native digital assets.	Asset eligibility, custody segregation, surveillance, jurisdictional capability.
Data / HIN Fabric	Consent, rights, compute-to-data, contribution participation.	Purpose, consent, privacy, contribution evidence and compensation basis.

DOMAIN INVARIANT

One user experience may aggregate value. Underlying legal, accounting, risk and settlement authorities remain separate.

Universal Ledger: Permission Ends Where Accounting Truth Begins

The canonical ledger is a separate authority from the Compliance Kernel. Permission determines whether an action may execute; the ledger establishes what financial state actually resulted. This distinction avoids a dangerous collapse in which authorization logic also becomes the balance system.

The source architecture specifies a double-entry ledger and six non-bypassable invariants. These invariants are useful because they turn financial correctness into system properties rather than application conventions.

Money is represented in whole minor units with exact arithmetic for rates where required, reducing ambiguity and avoiding floating-point balance behavior. Corrections are compensating entries rather than destructive edits, preserving historical traceability.

Invariant	Guarantee
Balance	Every journal balances per asset: total debits equal total credits.
Immutability	Postings are append-only; corrections use compensating entries.
Authority-required	Customer-impacting journals require signed Execution Authority.
Class bridges	Crossing account classes requires a named, disclosed bridge.
No commingling	Customer and corporate funds mix only through disclosed, audited bridges.
Idempotency	A repeated request replays to the same result without duplicating effect.

TRUTH RULE

The ledger establishes financial truth; the Evidence Vault establishes decision truth. Neither substitutes for the other.

Proof-Carrying Financial Events

Every consequential decision can be represented as a Proof-Carrying Financial Event. The object binds the final outcome to the identity state, authority, jurisdiction, compliance result, risk result, purpose evaluation, policy version, model version and other decision context used at the time.

This is not intended to put all sensitive information into an immutable global log. The event can carry hashes, references, proof identifiers, policy IDs and other evidence pointers while protected source data remains in the appropriate controlled system.

The core objective is reconstructability. A later auditor, regulator, operator or incident investigator should be able to determine which state and rules were authoritative when the system acted.

Proof-Carrying Financial Event

Every allow, condition, escalation and block can be sealed with the state that produced it.

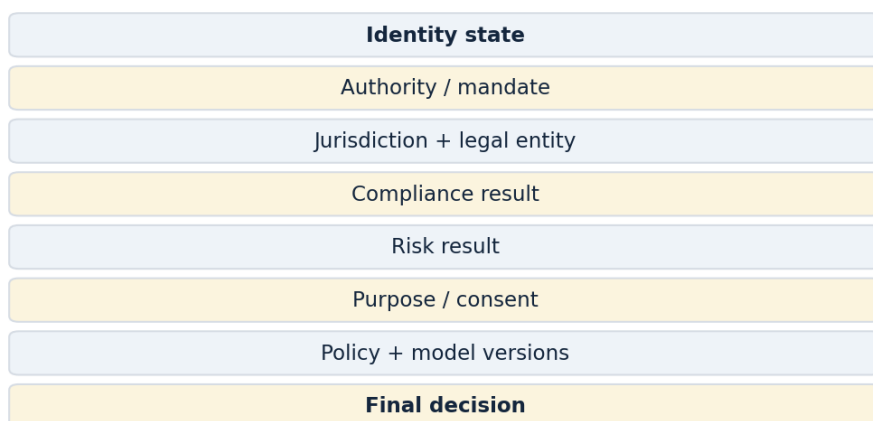


Figure 3. A proof-carrying event binds decision outcome to the state that produced it.

AUDIT QUESTION

Why was this transaction permitted, conditioned, escalated or blocked at that moment? The architecture is designed to answer mechanically.

Evidence Vault: Tamper-Evident Decision Reconstruction

The Evidence Vault is the terminal evidentiary layer of the authorization spine. The source architecture describes hash-chained, tamper-evident decision records that can be verified end-to-end, with divergence pointing to the location where the chain no longer matches expected history.

The Vault should capture both successful and unsuccessful pathways. A blocked transfer, rejected trade, revoked data use or deferred action can be just as important for demonstrating control effectiveness as a successful transaction.

The Evidence Vault is also where compliance becomes more than a runtime decision. Tests, policy versions, approvals, legal-review states, control outputs and transaction evidence can be associated with the operating state that produced the event.

- Preserve the authoritative policy and model versions used.
- Record control outcomes and required conditions.
- Reference evidence without exposing unnecessary sensitive raw data.
- Support transaction-level and control-level reconstruction.
- Make blocks, deferrals and manual-review paths auditable.
- Separate decision evidence from the canonical accounting ledger.

EVIDENCE DOCTRINE

If the system cannot later explain why it acted, it has not fully operationalized compliance.

Regulatory Digital Twin: What Is the System Legally Capable of Doing Here and Now?

The Regulatory Digital Twin is SFF 2.0's machine-readable model of legal and operational capability. It does not replace regulators, counsel, licenses or regulated partners. It encodes the approved result of those external realities into a form that product and control systems can enforce.

The Twin can evaluate capability by product, customer, jurisdiction, legal entity, license state, partner, control requirement, policy version and counsel review state. This allows a system to distinguish, for example, between a feature being implemented in software and that feature being authorized for a specific customer and corridor.

Operationally, the Twin reduces scattered country-specific branching logic. Instead of each product service inventing its own legal state, authoritative capability can be updated in one governed model and consumed by the control plane.

Regulatory Digital Twin

Legal and operational capability becomes a machine-readable chain from source requirement to reconstructable evidence.



Capability state is evaluated by product + customer + jurisdiction + legal entity + partner + license state + policy version + counsel review state.

Figure 4. The Regulatory Digital Twin maps source obligations into testable controls and reconstructable evidence.

CAPABILITY RULE

Software existence is not legal capability. A live feature requires a positively resolved product, jurisdiction, legal-entity and control state.

From Regulation to Evidence: The Control Traceability Chain

The source architecture expresses the Twin as a traceability chain: regulation -> requirement -> control -> policy -> software implementation -> test -> evidence -> regulatory report. Each transition answers a different governance question.

This chain is valuable because it makes change impact visible. A change in an external requirement can identify affected controls, policies, software paths, tests and evidence obligations. Conversely, a failed test can be traced back to the requirement and operational capability it protects.

The chain also separates legal interpretation from software enforcement. Legal and compliance owners determine the authoritative requirement and control state; engineering implements and tests that state. The software should not infer legal meaning independently.

Node	Primary question
Regulation / source	What external rule, license, partner condition or legal basis exists?
Requirement	What must the institution do or avoid?
Control	What operational mechanism satisfies the requirement?
Policy	How is the control expressed as machine-readable decision logic?
Software	Where is the policy enforced?
Test	How is control effectiveness verified?
Evidence	What proves the control and decision operated?
Report	How is the state reconstructed for oversight?

TRACEABILITY

A compliance architecture becomes materially stronger when a requirement can be traced all the way to executable control and evidence - and back again.

Machine-Readable Capability as a First-Class Object

A capability object answers whether a specific regulated function is available under a specific set of conditions. This is more precise than feature flags such as "crypto_enabled" or "international_payments=true." The relevant state can vary by customer, corridor, legal entity, partner, product, asset and policy version.

The Regulatory Digital Twin therefore functions as a capability registry as well as a traceability system. Product services can ask a bounded question and receive an authoritative answer rather than replicate legal logic locally.

Capability objects also support staged operations. The same product may exist in simulation, sandbox, partner pilot or production states, with different evidence and approval requirements.

Capability dimension	Illustrative state
Product	Cross-border transfer, investment action, custody, exchange, data participation.
Customer / account	Eligibility, identity status, risk class, mandate, approval state.
Jurisdiction / corridor	Source, destination, regional cell and applicable policy pack.
Legal entity / partner	Named operator, regulated partner and contractual basis.
License / counsel state	Confirmed, draft, research required, unavailable.
Control state	Required screenings, limits, disclosures and monitoring.
Operational state	Simulation, sandbox, partner pilot, production.
Evidence state	Tests, approvals, policy versions, audit artifacts and reports.

DESIGN CONSEQUENCE

There should be no single global switch for a regulated product family. Capability is contextual.

Policy Engine and Jurisdiction Packs

Regulatory rules should not be hard-coded throughout application services. The source architecture places them in versioned, testable Jurisdiction Packs evaluated by a central Policy Engine. A local pack operates on a default-deny, restrict-only basis: it may tighten the global baseline but should not silently loosen it.

Each product rule can carry a legal review state such as confirmed by counsel, draft or research required. Production capability is therefore tied to explicit legal review rather than inferred from the presence of code.

Versioning is essential. A transaction decision should record the exact policy version used, and policy updates should pass through governed change control and testing before they alter live capability state.

- Centralize policy evaluation; decentralize product execution.
- Version policy and preserve historical decision context.
- Require explicit legal-review state for regulated capability.
- Default unresolved capability to disabled.
- Allow regional policy to tighten a baseline without silent relaxation.
- Test policy as production logic, not documentation.

POLICY DOCTRINE

A rule that cannot be versioned, tested and evidenced is not yet an enforceable software control.

Legal Entity Resolution Is Part of Permission

The source architecture states that the software must never treat the brand itself as the legal actor by default. Every product, account, transaction, consent and execution should map to a named legal entity or regulated partner.

This is a subtle but important control dimension. The same user experience may combine accounts, payments, investments, exchange, custody, data participation and technology services that are operated by different legal entities or partners. The control plane must know which actor is responsible for the requested activity before Execution Authority can exist.

Legal-entity mapping therefore belongs inside jurisdiction and capability resolution, not in back-office reporting after execution.

Activity family	Required resolution
Accounts / payments	Named banking, payment or sponsor-partner legal actor.
Investment / portfolio	Named advisory, management, broker or execution actor where applicable.
Exchange / custody	Named exchange, custody or digital-asset operator where applicable.
Data / HIN	Named controller, processor, clean-room or market operator as applicable.
Technology / AI	Named technology service with bounded proposal authority, not financial principal authority.

KERNEL REQUIREMENT

An ActionIntent without a resolved legal actor cannot receive Execution Authority.

Sovereign Cells: Compliance by Architecture, Not Convention

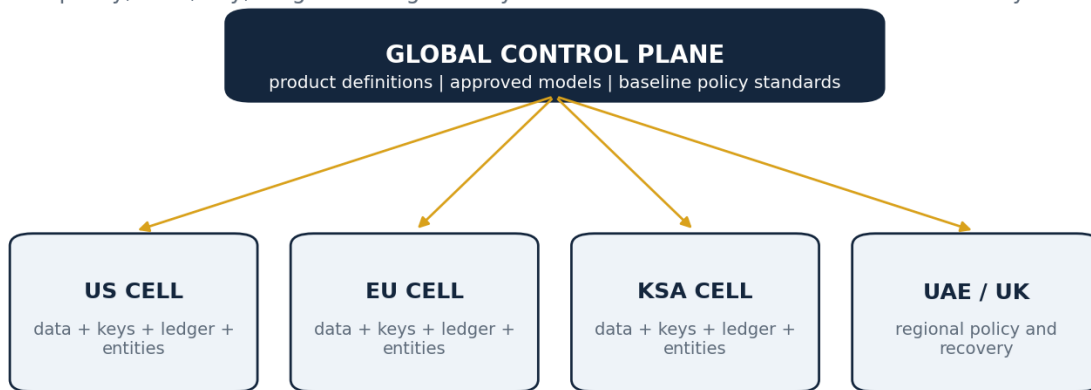
Sovereign Cells are regional deployments that enforce regulatory, data-residency, cryptographic and operational boundaries structurally. Each cell can own customer data, encryption keys, ledger partitions or local ledgers, legal-entity mappings, policy packs, approved models and recovery procedures.

A global control plane may distribute approved product definitions, model versions and baseline policy standards, but it should not require unrestricted access to every individual's raw information. This supports regional autonomy while preserving a common authorization grammar.

The failure model is equally important: a failure in one cell should not require the entire global system to fail. Regional restrictions or emergency controls can be applied without granting a global application service unlimited authority.

Sovereign Cells

Regional policy, data, key, ledger and legal-entity boundaries are structural rather than merely conventional.



A cell can tighten the global baseline. It must not silently loosen it.

Figure 5. Sovereign Cells make jurisdictional, data and key boundaries structural.

DEPLOYMENT PRINCIPLE

Regulatory segmentation is stronger when it is represented in infrastructure, keys, data and policy boundaries - not only in application if-statements.

Agent Capability Tokens: Proposal Authority Without Execution Credentials

AI agents operate under explicit capability boundaries. The source architecture describes Agent Capability Tokens that specify what an agent may propose, what it is forbidden from doing, per-transaction and daily limits, allowed account classes, forbidden data categories, expiry and revocation.

These constraints are enforced in infrastructure rather than inferred from prompts. The agent can therefore reason broadly while its actual action surface remains narrow. A model jailbreak, hallucination or overconfident recommendation does not automatically translate into a financial event.

Capability tokens also make responsibility legible. An investment research agent, payment-routing agent and subscription agent can have different proposal surfaces even if they share underlying model infrastructure.

Capability field	Control purpose
Allowed proposal types	Defines the categories of ActionIntent the agent may emit.
Explicit prohibitions	Blocks classes of action regardless of model reasoning.
Amount / frequency limits	Caps economic scale and cadence.
Account / asset classes	Restricts where proposals may apply.
Data categories	Restricts sensitive information available for reasoning.
Expiry / revocation	Ensures capability can end without model cooperation.
Model / version binding	Limits capability to approved model lifecycle state.

AGENT RULE

AI agents hold proposal capability, not execution credentials.

Cross-Border Transfer: Permission Across a Corridor

A cross-border transfer illustrates why Proof of Permission must be richer than an API call. The action crosses identity, customer authority, source and destination jurisdictions, legal entities, FX, sanctions and transaction monitoring, beneficiary validation, liquidity, payment rail selection, disclosures, settlement and reconciliation.

The routing engine should score only routes that are already permitted. A cheaper or faster path that fails jurisdictional, partner, risk or compliance requirements is not an eligible route. The selected path is then executed under a bounded Execution Authority and reconciled against partner and bank evidence.

The Evidence Vault can preserve the quotes considered, route decision, beneficiary checks, policy state, required disclosures and settlement evidence without giving any single provider the ability to become canonical permission or accounting authority.

Stage	Proof / control focus
Intent	Sender, beneficiary, amount, currencies, purpose and idempotency.
Kernel	Identity, authority, source/destination jurisdiction, compliance, risk and purpose.
Twin	Corridor, legal-entity and partner capability state.
Routing	Evaluate only eligible rails for cost, speed, liquidity and reliability.
Execution	Use signed authority for the selected bounded route.
Ledger / reconciliation	Record canonical postings and reconcile external settlement.
Evidence	Store decision, quote, policy and settlement proofs.

CORRIDOR PRINCIPLE

A transfer route can be technically reachable yet legally or operationally ineligible. Eligibility is resolved before optimization.

AI Investment Proposal: Intelligence Under Deterministic Risk

An AI investment system can discover a market opportunity, estimate expected behavior and propose a portfolio action. Proof of Permission prevents that intelligence layer from directly becoming trading authority.

The ActionIntent captures the proposed instrument, amount, account, strategy context, customer mandate and expected risk. Authority checks the mandate and any required user approval. Jurisdiction and compliance resolve whether the product and activity are available for that customer and legal entity. Risk determines whether exposure fits deterministic limits. Model State confirms that the model is approved for the product and jurisdiction.

Only after these dimensions resolve can a signed Execution Authority reach the controlled execution infrastructure. Venue selection and order execution remain policy-bounded, and final positions are recorded in the appropriate ledger or custody authority rather than inside the AI service.

- Agentic Capital Mesh searches and proposes.
- Deterministic Risk Engine outranks strategy allocation decisions.
- Model State gates the economically consequential model lifecycle.
- Mandates and user approval establish bounded authority.
- Execution policy and kill switches remain outside the model.
- Ledger/custody state is canonical after execution.

AI-FINANCE PRINCIPLE

A better model may improve proposals. It does not receive a broader financial mandate by virtue of being better.

Data-Dependent Economic Action: Purpose and Consent in the Authorization Chain

Proof of Permission also applies when a financial or economic action depends on sensitive information. The Personal Economic Graph may contain permission-aware relationships across accounts, goals, expenses, work, wellness or other authorized data. The system must decide not only whether the information is relevant, but whether it may be used for this purpose by this agent or model.

Purpose proof evaluates the authorized use. Consent can be treated as an explicit subproof where required. The action may operate on derived attributes, proofs or clean-room outputs rather than exporting raw source records. The resulting event can preserve consent references and purpose state without exposing unnecessary underlying data.

This keeps privacy and compliance inside the same authorization grammar as money movement rather than treating data governance as a separate afterthought.

Control question	Architecture response
Is the fact relevant?	Personal Economic Graph provides permission-aware context.
May this agent/model use it?	Purpose, consent, permitted-agent and permitted-model metadata gate use.
Must raw data move?	Prefer protected vault / clean-room computation and proofs where possible.
What economic action follows?	Translate into an ActionIntent and re-enter the normal six-proof chain.
What must be evidenced?	Purpose basis, consent reference, policy/model version and final outcome.

PRIVACY PRINCIPLE

Knowing something never implies permission to use it.

Simulation, Testing and Release Gates

The source architecture is simulation-first. A regulated capability should not become live merely because the software path exists. Jurisdiction Packs, provider bindings, legal-review state, tests, operational evidence and explicit approval all participate in production capability.

This is particularly important for AI systems because model behavior and control behavior must be tested separately. A model can be evaluated for proposal quality while the Kernel, Risk Engine, Execution Authority, ledger invariants and evidence reconstruction are independently validated.

Testing should include both positive and negative paths. A mature control plane proves that it can allow valid actions and reliably refuse invalid, stale, over-limit, unauthorized, unsupported or poorly evidenced actions.

Test family	What it should prove
Policy tests	Versioned rules produce expected allow/condition/escalate/block outcomes.
Authorization tests	No execution path bypasses ActionIntent + Execution Authority.
Ledger tests	Double-entry, idempotency, class bridges and no-commingling invariants hold.
Model tests	Only approved models can participate in economically consequential workflows.
Jurisdiction tests	Unsupported or unresolved capability remains disabled.
Evidence tests	Every decision can be reconstructed from preserved state and references.
Chaos / failure tests	Provider, cell, model and service failures fail safely without creating authority.

RELEASE RULE

Production-capable code is not production authorization.

Failure Modes the Architecture Is Designed to Prevent

Proof of Permission is most useful when evaluated against failure modes rather than ideal flows. The control model assumes that models can be wrong, providers can be unavailable or inconsistent, users can retry, policies can change, data can become stale, and regional capability can be revoked.

Because authority is explicit, the system can fail closed without requiring every downstream component to understand the entire legal and risk context. The absence of valid Execution Authority becomes a simple and enforceable reason not to move money or change protected economic state.

The same approach reduces blast radius. A compromised or malfunctioning agent cannot silently acquire wallet authority; a provider success response cannot mint money; a control-room dashboard cannot become an execution plane; and a local jurisdictional restriction can disable a capability without globally rewriting product logic.

Failure mode	Control response
AI hallucination / jailbreak	Proposal is bounded by capability token and must still clear the Kernel.
Stale KYC / consent	Require refresh, step-up, defer or block.
Policy change	New version updates capability; historical events retain old decision state.
Duplicate request	Idempotency replays the same result without duplicate effect.
Provider returns success unexpectedly	Provider evidence cannot bypass Execution Authority or ledger reconciliation.
Jurisdiction becomes unavailable	Capability state disables affected product path.
Regional outage	Cell failure is isolated; global system need not grant emergency bypass authority.

ADVERSARIAL PRINCIPLE

No single model, provider, dataset, operator, service flag or UI state should be able to silently create financial authority.

Regulator and Auditor Reconstruction as an Architectural Requirement

A compliance-native system should be designed so that oversight questions can be answered from durable state rather than reconstructed manually from scattered logs and institutional memory.

For a given transaction or blocked action, an investigator should be able to identify the initiating intent, relevant customer and identity state, authorizing mandate or approval, governing legal actor and jurisdiction, control and risk results, policy/model versions, capability state, execution authority, ledger entries, external settlement evidence and final outcome.

The Regulatory Digital Twin expands this from transaction reconstruction to control reconstruction. An institution can trace a rule to the requirement, policy, implementation, test, evidence and report that operationalize it.

- Transaction reconstruction: what happened and why?
- Capability reconstruction: why was this product available to this customer here?
- Control reconstruction: which requirement did this test and policy satisfy?
- Change reconstruction: what policy or capability changed and when?
- Incident reconstruction: which authority boundary failed, if any?

AUDITABILITY

The design target is not a bigger log archive. It is a mechanically coherent evidence graph linking requirement, decision, execution and financial truth.

Change Management for Policy, Models and Capability State

Compliance-native infrastructure must manage change as carefully as transaction execution. Regulations, counsel interpretations, partner agreements, product scopes, model versions, risk limits and operational readiness can all change independently.

The architecture therefore benefits from versioned policy, explicit legal-review state, model lifecycle governance, machine-readable capability state and evidence of the tests and approvals that accompanied each activation. A change should be attributable to an authorized human or governance process rather than to an autonomous model.

Rollback should also be first-class. If a new model, policy or provider binding fails qualification, the system should be able to restore the prior approved state without rewriting historical evidence.

Change type	Required control
Policy update	Version, test, legal/compliance ownership, activation evidence.
Model update	Validation, product/jurisdiction approval, monitored lifecycle state.
Risk-limit update	Authorized owner, bounded scope, effective time and evidence.
Provider binding	Contract/operating scope, credential state, qualification and fallback.
Jurisdiction capability	Legal-review state, named entity/partner, controls and effective date.
Emergency restriction	Rapid tighten/disable path; no equivalent silent relax path.

GOVERNANCE RULE

AI may assist analysis of control changes. It should not autonomously grant itself broader legal, model, risk or execution authority.

Why This Architecture Matters to Banks, Regulators and Enterprise Partners

For regulated institutions, the value of agentic AI depends on whether it can be integrated without dissolving established accountability. Proof of Permission offers a pattern in which AI capability can sit above a deterministic, evidence-producing control plane rather than inside the authority boundary itself.

For banks and service partners, the model clarifies which system owns proposal, eligibility, execution, accounting and evidence. For regulators and auditors, it creates explicit traceability from rule to control to transaction. For enterprise buyers, it supports bounded integration with provider and jurisdiction-specific capability rather than requiring a global trust assumption.

The architecture is not a substitute for institutional governance. It is a mechanism for turning approved governance into executable, testable and reconstructable controls.

Audience	Potential value
Bank / regulated financial institution	Introduce AI decision support while preserving deterministic transaction authority and canonical ledger control.
Regulator / supervisor	Inspect machine-readable capability, policy versions and transaction-level decision evidence.
Audit / risk function	Trace requirements to controls, tests, evidence and historical transaction outcomes.
Fintech / BaaS partner	Integrate through bounded domain interfaces without becoming the platform's global permission authority.
Enterprise buyer	Deploy regionally with named legal actors, explicit capability and auditable control state.

INSTITUTIONAL THESIS

The strongest AI-finance architecture may be the one that increases machine intelligence while making human, legal and financial authority more explicit.

Twelve Principles of Compliance-Native Financial Infrastructure

1. **Proposal is not execution.** All consequential activity begins as an ActionIntent.
2. **Permission is explicit.** Execution authority is never inferred from technical capability.
3. **Six proofs are foundational.** Identity, Authority, Jurisdiction, Compliance, Risk and Purpose must resolve.
4. **AI does not hold unrestricted financial authority.** Agents operate under bounded proposal capabilities.
5. **Policy is versioned software control.** Rules are testable, governable and reconstructable.
6. **Legal capability is contextual.** Product availability depends on customer, jurisdiction, entity, partner and control state.
7. **Risk outranks opportunity.** Economic attractiveness cannot override deterministic exposure limits.
8. **Data possession is not purpose permission.** Purpose and consent constrain information use.
9. **Execution is signed and scoped.** Only short-lived Execution Authority can authorize a bounded action.
10. **Accounting truth is canonical.** Ledgers and custody/chain authorities, not applications or agents, establish financial state.
11. **Evidence is part of the transaction system.** Every outcome can be reconstructed from proof-carrying state.
12. **Production is an authorization state, not a code state.** Simulation, testing, legal review, provider scope and human approval remain distinct from implementation.

SUMMARY

Proof of Permission is a systems doctrine for keeping intelligence, permission, execution, accounting and evidence separate - and then connecting them through explicit, verifiable objects.

From Compliance Checks to a Permissioned Financial Operating Layer

The central claim of Proof of Permission is architectural: AI-driven finance should not rely on the model, application service, provider or user interface to carry implicit authority. Every consequential proposal should enter a deterministic permission chain before it can affect money, investments, digital assets or sensitive information.

The six foundational proofs provide the grammar. ActionIntent provides the boundary object. The Compliance Kernel resolves permission. Signed Execution Authority creates a narrow executable capability. Financial fabrics preserve domain segregation. The canonical ledger establishes financial truth. Proof-Carrying Financial Events and the Evidence Vault preserve why the system acted. The Regulatory Digital Twin turns legal and operational capability into machine-readable, testable state.

This pattern allows AI capability to improve without silently expanding legal or financial power. It also gives institutions a path toward more autonomous financial systems in which control becomes more explicit as intelligence becomes more powerful.

FINAL THESIS

The future of compliant financial AI is not unrestricted autonomy. It is bounded agency operating inside an architecture that can prove who authorized what, under which law and controls, within which risk and purpose limits, and with what evidence.

Source Basis

- Solstice - Global AI Financial Operating System with Pyramid Coin (PYR) & Pyramid Exchange: Architecture & Feature Reference. Version 1.0, 13 August 2026. Source for the original SFF authorization spine, six-proof model, Jurisdiction Packs, ledger invariants, Agent Capability Tokens, Evidence Vault and Sovereign Cells.
- Solstice SFF 2.0 - Five First-Class Architecture Components. August 2026. Source for the Regulatory Digital Twin and its regulation -> requirement -> control -> policy -> software -> test -> evidence -> report mapping, plus machine-readable capability state.
- Solstice Comprehensive Product, Architecture, API, Compliance & Growth Systems Report - Sovereign Financial Fabric 2.0. Version 2.0, 14 August 2026. Source for expanded proof dimensions, deterministic outcomes, legal-entity requirements and regulatory operating-system detail.
- SunRey Comprehensive Business Plan. September 2026. Source for current SunRey naming and the non-negotiable control doctrine separating AI proposal, policy permission, deterministic risk, user approval where required, controlled execution and ledger truth.
- SunRey Blockchain & Universal Economic Exchange - Technical Architecture and Novel Market Design. August 2026. Context for machine-readable identity, rights, policy, provenance, jurisdiction and evidence as economic state.

Terminology Note

Earlier architecture documents use the working names Solstice and Pyramid. Current public-facing terminology uses SunRey, SunRey Coin, MoonRey Coin and SunRey Exchange. This paper updates the brand language while preserving the technical meaning of the authorization architecture.